

GUI-Based Multi-Client Chat Application Using TCP

Assignment 6 Report

Name: Omer Osman Omer Elhuseein

Roll Number: 324106410070

Topology: single,5 (h1=Server, h2–h5=Clients)

GUI Framework: Python Tkinter

Extended from: Assignment 5

1. Objective

This assignment converts the terminal-based TCP chat application from Assignment 5 into a graphical desktop application using Python Tkinter. The server.py is reused unchanged. The new client_gui.py replaces client.py with a full GUI supporting login, broadcast messaging, private messaging, and a live online users panel.

2. GUI Design

2.1 Login Window

A simple 300×180 white window with a username entry box, a Connect button, and a status label. Validates that the username is non-empty and space-free. Attempts a TCP connection to 10.0.0.1:5000 and opens the Chat Window on success.

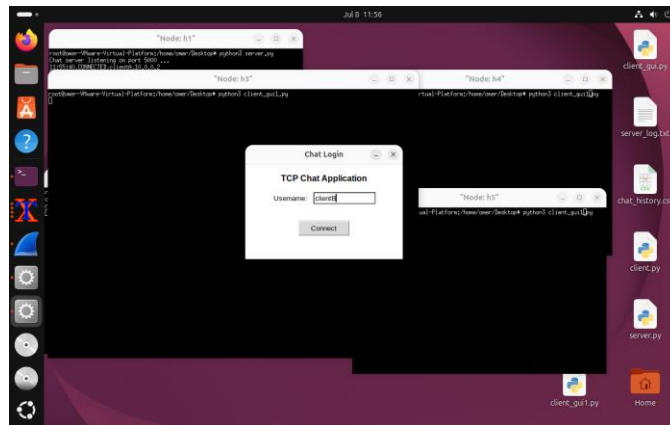


Fig 1: Login window — username entry and Connect button

2.2 Chat Window

A 700×500 resizable window with four sections:

- (1) Status bar — shows username, server IP, and Online/Offline status.
- (2) Scrollable message area — displays all received messages in black on white.
- (3) Online Users panel (right) — live listbox; double-click to auto-fill /msg.
- (4) Input row — message entry + Send button + Disconnect button always visible.

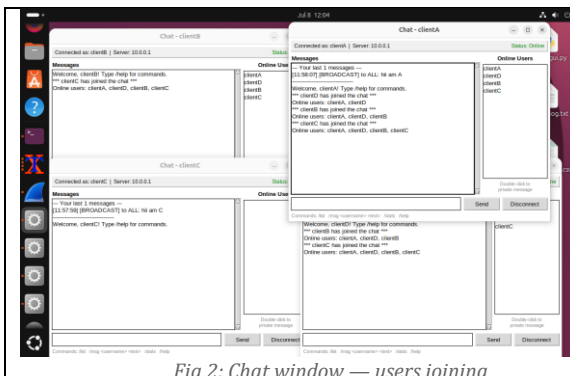


Fig 2: Chat window — users joining

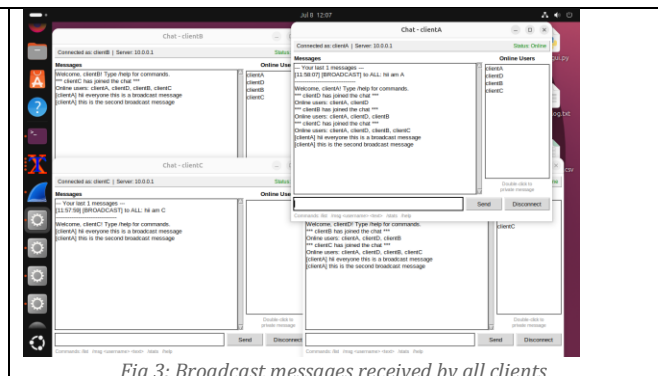


Fig 3: Broadcast messages received by all clients

3. System Architecture

Topology: sudo mn --topo single,5 — h1=Server, h2–h5=Clients.

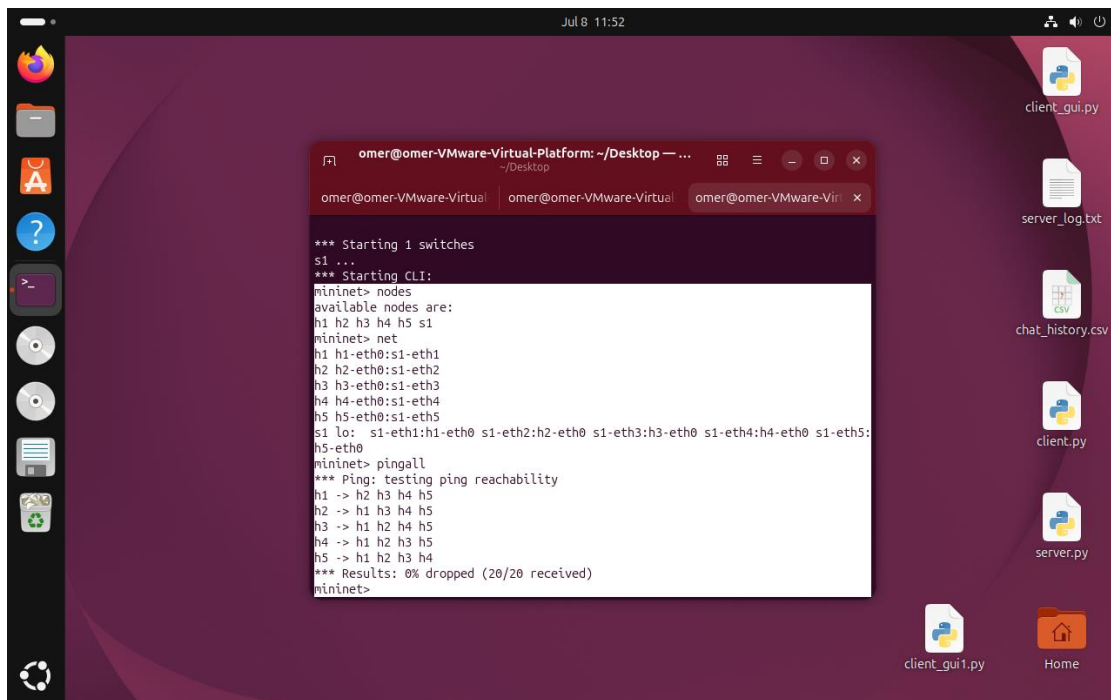


Fig 4: nodes, net, pingall — 0% packet loss (20/20 received)

The architecture has two layers: (1) Server layer — `server.py` from Assignment 5, unchanged, handles threading, broadcasting, private routing, and logging. (2) GUI client layer — `client_gui.py` wraps the same socket logic in a Tkinter interface. A background daemon thread runs `_receive_loop()` and schedules all GUI updates via `win.after(0, ...)` to keep the GUI responsive.

4. Components Reused from Assignment 5

Component	Reused?	Notes
<code>server.py</code>	Yes — unchanged	Handles all clients, broadcast, private msg, logging
TCP socket logic	Yes	Same connect/send/rcv pattern in <code>client_gui.py</code>
<code>/list, /msg, /stats</code> commands	Yes	Sent as plain text; server processes identically
<code>chat_history.csv</code>	Yes	Server still logs all messages
<code>server_log.txt</code>	Yes	Server still logs connections/disconnections
<code>client.py</code> (terminal)	Replaced	Replaced by <code>client_gui.py</code> with Tkinter GUI

5. GUI Design Decisions

Grid layout was chosen over `pack()` for the chat window to guarantee the Send and Disconnect buttons are always visible and the message area fills available space correctly. The Online Users listbox has a fixed minimum width of 150px so it never gets hidden. A `minsize(600,420)` prevents the window from being shrunk to the point where widgets disappear. All socket operations run in a background thread; `win.after(0, ...)` is used for all GUI updates to avoid race conditions between the receive thread and the Tkinter event loop.

6. Testing Results

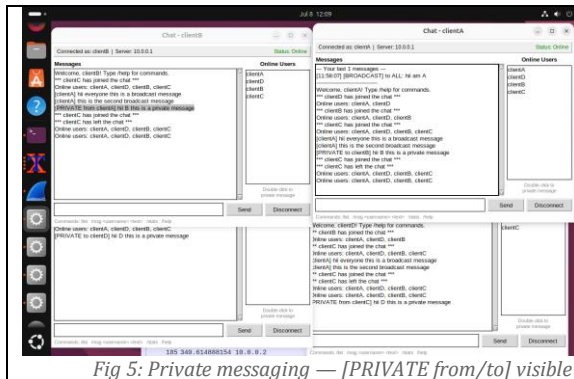


Fig 5: Private messaging — [PRIVATE from/to] visible

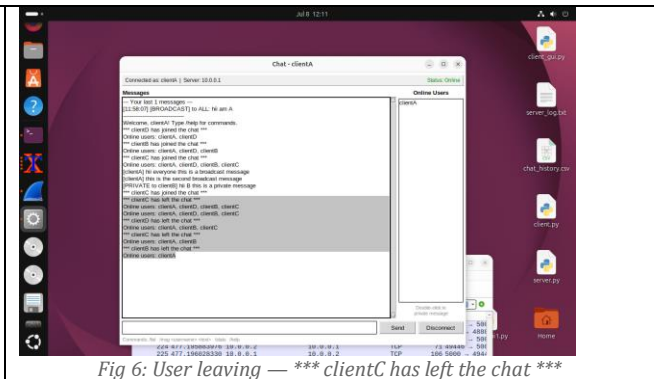


Fig 6: User leaving — *** clientC has left the chat ***

All features tested successfully with 4 simultaneous GUI clients (clientA–clientD):

Feature	Result
Login & connect	Pass — all 4 clients connected via GUI
Broadcast messaging	Pass — messages appeared in all windows
Private messaging (/msg)	Pass — only recipient received message
Online user list (auto-refresh)	Pass — updated on every join/leave
Chat history on reconnect	Pass — last messages shown on login
User join notification	Pass — *** username has joined the chat ***
User leave notification	Pass — *** username has left the chat ***
Disconnect button	Pass — closes socket, returns to login window

7. Wireshark Verification

TCP traffic captured on interface s1-eth1. Filter: tcp.port == 5000

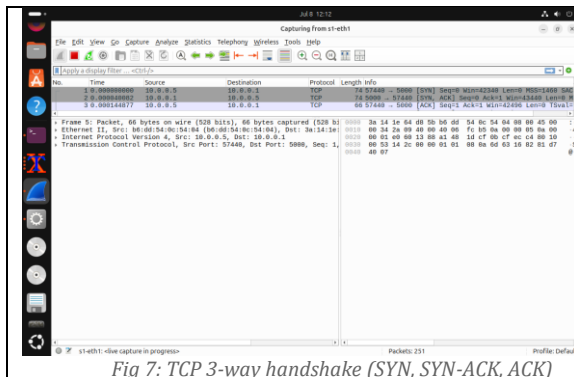


Fig 7: TCP 3-way handshake (SYN, SYN-ACK, ACK)

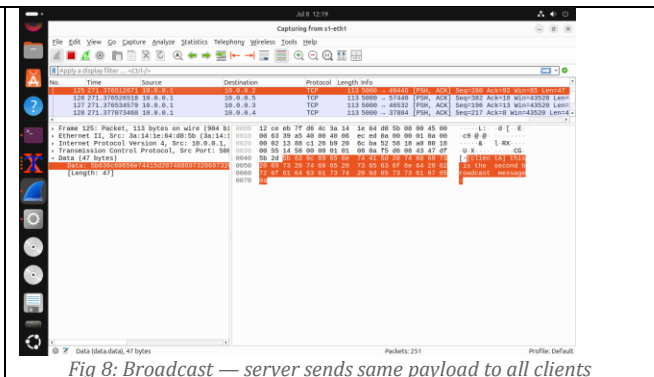


Fig 8: Broadcast — server sends same payload to all clients

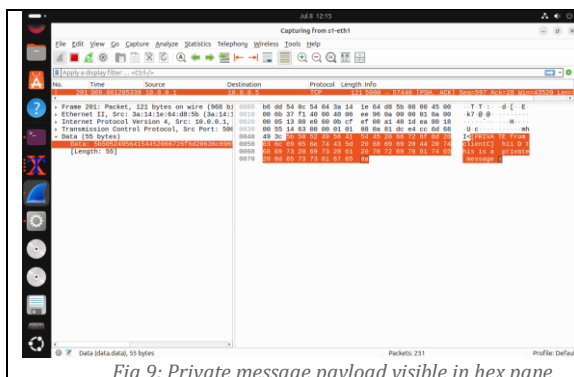


Fig 9: Private message payload visible in hex pane

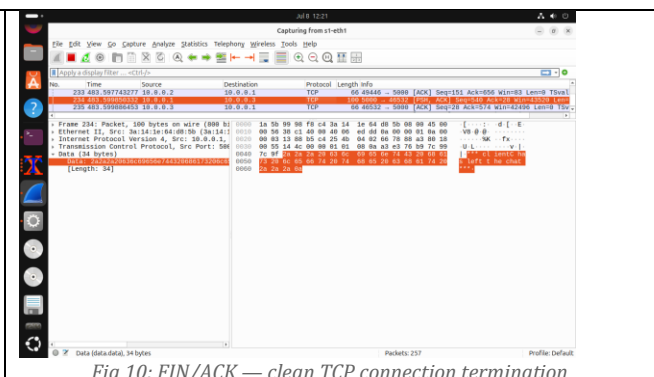


Fig 10: FIN/ACK — clean TCP connection termination

Wireshark confirmed: (1) each GUI client connection triggered a SYN/SYN-ACK/ACK handshake; (2) broadcast messages appeared as identical PSH/ACK packets sent from the server (10.0.0.1) to multiple destination ports simultaneously; (3) private message payloads (e.g. "[PRIVATE from clientC] hii D this is a private message") were visible in the hex pane, sent only to the target client; (4) clicking Disconnect triggered a clean FIN/ACK teardown.

8. Reflection Answers

Why is server-side routing necessary for private messaging?

GUI clients have no direct socket to each other — all connections go through the server. Only the server holds the username-to-socket mapping, so it must intercept /msg commands and forward them exclusively to the target socket.

How does maintaining user state improve the application?

Storing username, IP, login time, and status enables the live /list panel, join/leave notifications, private message routing by name, and chat history recall on reconnect — none of which are possible without per-client state.

How would the application scale to 100 users?

Switch from thread-per-client to asyncio for lower memory overhead. Replace CSV logging with SQLite for thread-safe history. Use a message queue to decouple receive and broadcast.

Which Wireshark packets verified private messaging?

Frame 201 (Fig 9) — a PSH/ACK from server (10.0.0.1) to h5 (10.0.0.5) — showed "[PRIVATE from clientC] hii D this is a private message" in the hex/ASCII pane, confirming the server forwarded it only to the intended recipient.

What improvements would you implement next?

(1) Chat rooms/channels. (2) TLS encryption for private messages. (3) File transfer. (4) Message timestamps displayed in the GUI. (5) User authentication with passwords.

9. Conclusion

Assignment 6 successfully converted the terminal-based Assignment 5 chat application into a fully functional Tkinter GUI. The server was reused without modification. The GUI client implements a login window, scrollable chat area, live online users panel, private and broadcast messaging, and clean disconnect handling — all while keeping the socket logic in a background thread so the interface remains responsive. Wireshark captures independently verified all four key network behaviours: connection, broadcast, private messaging, and disconnection.